Министерство образования Республики Беларусь
Учреждение образования
«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ»

Кафедра информатики

ОТЧЕТ
О ЛАБОРАТОРНОЙ РАБОТЕ № 3
ВЗАИМОДЕЙСТВИЕ ПРОЦЕССОВ: ОБМЕН ДАННЫМИ

по дисциплине «Операционные среды и системное программирование»

Выполнил ст. гр. 453504 Л. Д. Масалова

Проверил старший препод. каф. Информатики А. Д. Сыс

Минск 2026

1 ЦЕЛЬ РАБОТЫ

Изучить механизмы межпроцессного взаимодействия (IPC – Inter-Process Communication) в операционной системе Windows и получить практические навыки организации обмена данными между независимыми процессами.

В ходе выполнения лабораторной работы необходимо освоить:

– создание нескольких взаимодействующих процессов;

– передачу данных между процессами;

– разработку собственного простого протокола обмена сообщениями;

– использование каналов (Pipe);

– использование именованных каналов (Named Pipe);

– использование разделяемой памяти;

– применение объектов синхронизации при работе с разделяемой памятью;

– использование Mailslot;

– согласование работы процессов;

– обработку отключения или аварийного завершения одного из участников;

– предотвращение повреждения совместно используемых данных;

– измерение характеристик IPC;

– корректное освобождение системных ресурсов.

2 ЗАДАНИЕ

Индивидуальный вариант №15: Конвейер с обработкой отказа.

Механизм: Named Pipe.

Процессы: Generator → Processor → Validator → Collector.

Каждое сообщение имеет requestId.

Processor для части сообщений специально должен возвращать статус ERROR.

Validator:

– принимает корректные результаты;

– ошибочные возвращает на повторную обработку.

Разрешается не более двух повторов одного задания.

3 РЕЗУЛЬТАТ РАБОТЫ

3.1 Описание предметной области и задачи

Разработанный программный комплекс представляет собой распределенный конвейер из четырех изолированных процессов

операционной системы Windows, обменивающихся сообщениями в дуплексном режиме через виртуальную файловую систему именованных каналов (Named Pipe File System – NPFS). Использование глобальных переменных исключено, так как каждый процесс исполняется в собственном изолированном виртуальном адресном пространстве.

В состав комплекса входят следующие компоненты:

– Диспетчер (Master-процесс): выполняет первичную инициализацию, порождает дочерние процессы с помощью функции CreateProcessA() с флагом CREATE_NEW_CONSOLE (обеспечивая визуализацию каждого участника в отдельном окне терминала) и осуществляет барьерное ожидание их завершения через WaitForMultipleObjects();

– Генератор (Generator): процесс-источник данных. Подключается ко входному каналу обработчика, формирует поток из 172 заданий с уникальными идентификаторами requestId, инкрементируемыми метками времени и отправляет терминальное сообщение MSG_STOP по окончании генерации;

– Обработчик (Processor): выполняет бизнес-логику обработки. Имеет два независимых сервера входящих каналов (для новых задач от Generator и для повторных задач от Validator). Для каждого 3-го задания симулирует ошибку вычислений (MSG_RESULT_ERR), а для задания №72 генерирует фатальный сбой на всех итерациях;

– Валидатор (Validator): центральный узел отказоустойчивости. Принимает результаты от Processor. Успешные сообщения (MSG_RESULT_OK) транслируются в Collector. При получении ошибки MSG_RESULT_ERR проверяется счетчик попыток: если retryCount < 2, счетчик увеличивается, сообщение маркируется как MSG_TASK и возвращается в Processor по выделенному каналу повторов; при исчерпании лимита (retryCount == 2) сообщение помечается как фатальный сбой и сбрасывается в Collector;

– Коллектор (Collector): терминальный узел конвейера. Принимает финальные результаты, ведет учет успешно обработанных и отброшенных задач, фиксирует прохождение сигнала MSG_STOP и формирует итоговую статистику.

3.2 Схема взаимодействия процессов

Схема маршрутизации сообщений и топологии именованных каналов представлена на рисунке 1.

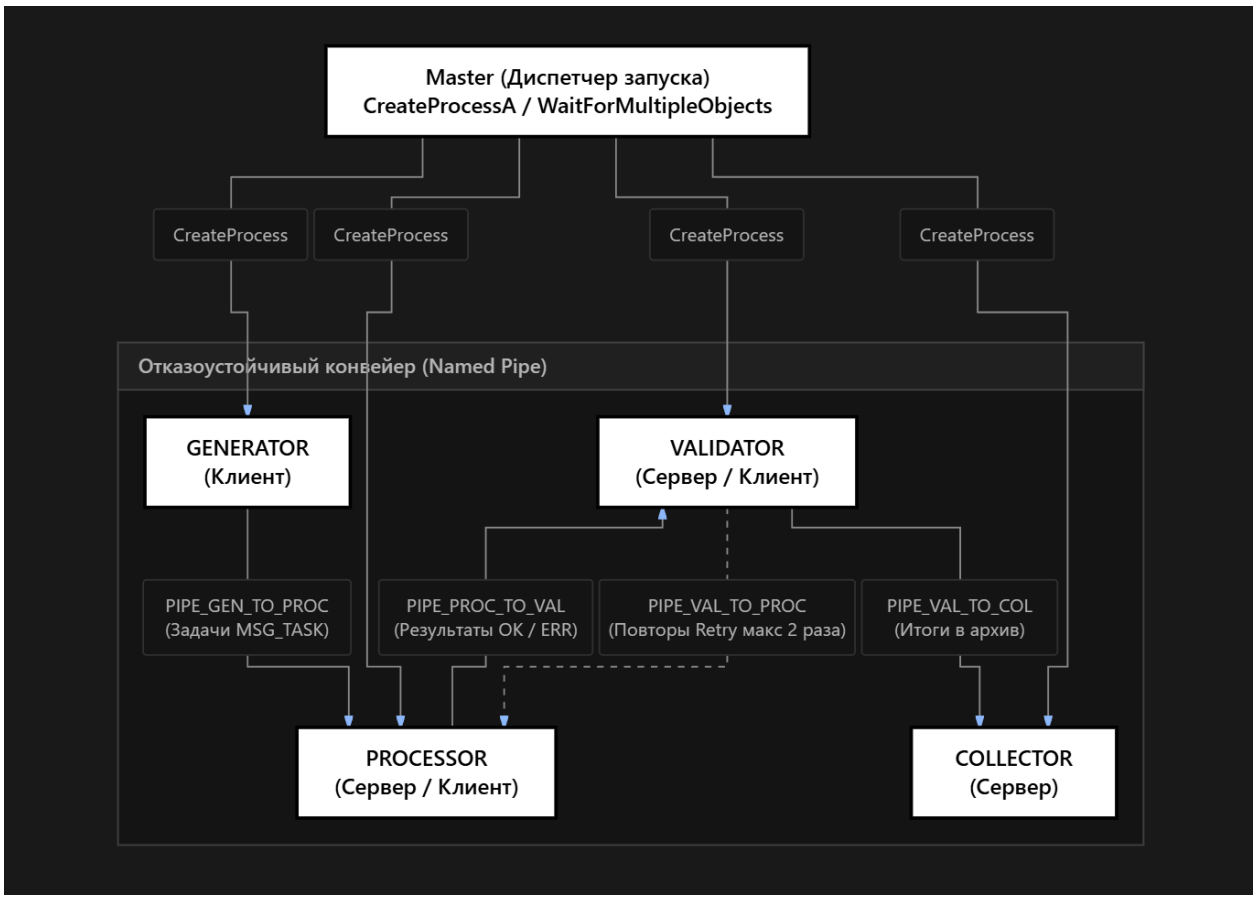


Рисунок 1 – Топология именованных каналов и маршрутизация сообщений

3.3 Формат сообщений и таблица типов протокола

Для межузлового обмена разработан бинарный протокол фиксированной длины без использования сырых указателей, что исключает ошибки сегментации памяти при межпроцессной передаче. Заголовок и тело упакованы с директивой #pragma pack(push, 1) для исключения межбайтового выравнивания.

Структура сообщения включает следующие поля:

– version (unsigned int) – версия используемого протокола (значение 1);

– messageType (unsigned int) – числовой идентификатор команды или статуса;

– sourcePid (unsigned int) – системный PID процесса-отправителя для трассировки;

– requestId (unsigned int) – сквозной идентификатор задания;

– retryCount (unsigned int) – текущий счетчик совершенных повторных попыток (0, 1 или 2);

– payloadSize (unsigned int) – фактическая длина полезных данных в буфере;

– payload[256] (char) – байтовый массив полезной нагрузки.

Таблица типов сообщений протокола представлена в таблице 1.

Таблица 1 – Спецификация типов сообщений протокола конвейера

Тип сообщения	Отправитель	Получатель	Содержимое полезной нагрузки (Payload)	Ожидаемая реакция
MSG_TASK	Generator	Processor	Исходные параметры задания (строка данных)	Обработка и генерация статуса
MSG_TASK (Retry)	Validator	Processor	Параметры задания с инкрементом retryCount	Повторная обработка в приоритетном порядке
MSG_RESULT_OK	Processor	Validator	Текст успешного результата вычислений	Пересылка в Collector
MSG_RESULT_ERR	Processor	Validator	Описание возникшей ошибки обработки	Валидация и принятие решения о повторе
MSG_RESULT_ERR (Drop)	Validator	Collector	Сообщение об исчерпании лимита 2 повторов	Фиксация в статистике сбоев
MSG_STOP	Generator / Validator	Processor / Collector	Пустой буфер (payloadSize = 0)	Каскадная остановка конвейера

3.4 Перечень использованных функций Windows API

В ходе реализации программы были задействованы следующие системные функции Windows API:

– CreateNamedPipeA() – создание экземпляра именованного канала на стороне сервера с режимом передачи сообщений (PIPE_TYPE_MESSAGE);

– ConnectNamedPipe() – перевод серверного процесса в режим ожидания подключения клиента к каналу;

– WaitNamedPipeA() – проверка доступности канала и блокирующее ожидание его открытия сервером со стороны клиента;

– CreateFileA() – открытие существующего именованного канала на стороне процесса-клиента;

– ReadFile() и WriteFile() – синхронное блочное чтение и отправка структур сообщений через дескриптор канала;

– PeekNamedPipe() – неблокирующая проверка наличия входящих данных в канале повторов без извлечения;

– DisconnectNamedPipe() – принудительное отключение клиентского конца канала сервером для корректного разрыва сессии;

– CloseHandle() – закрытие файловых дескрипторов каналов, процессов и первичных потоков;

– CreateProcessA() – порождение изолированных дочерних процессов в отдельных окнах консоли (CREATE_NEW_CONSOLE);

– WaitForMultipleObjects() – барьерная синхронизация Master-процесса при ожидании завершения всех дочерних процессов;

– GetLocalTime() – считывание системного времени с миллисекундной точностью для формирования журнала;

– GetCurrentProcessId() – определение уникального идентификатора PID текущего процесса.

– SetConsoleCP() и SetConsoleOutputCP() – переключение кодировки консоли ввода/вывода на UTF-8 для корректной работы с кириллицей.

3.5 Основные фрагменты исходного кода

В листинге 1 представлены ключевые функции низкоуровневого управления именованными каналами, реализующие создание серверного экземпляра и подключение со стороны клиента. Функция CreateServerPipe() использует системный вызов CreateNamedPipeA() для инициализации дуплексного канала в режиме передачи сообщений PIPE_TYPE_MESSAGE, что гарантирует сохранение границ структур при чтении. Функция ConnectClientPipe() реализует клиентскую сторону: она ожидает доступности канала через WaitNamedPipeA() и подключается к нему посредством стандартного вызова CreateFileA().

```
inline HANDLE CreateServerPipe(const char* pipeName) {
    HANDLE hPipe = CreateNamedPipeA(
        pipeName, PIPE_ACCESS_DUPLEX,
        PIPE_TYPE_MESSAGE | PIPE_READMODE_MESSAGE | PIPE_WAIT,
        1, sizeof(PipeMessage) * 16, sizeof(PipeMessage) * 16,
        NMPWAIT_USE_DEFAULT_WAIT, NULL
```

```
        );
        if (hPipe == INVALID_HANDLE_VALUE)
PrintError("CreateNamedPipe");
        return hPipe;
    }

    inline HANDLE ConnectClientPipe(const char* pipeName) {
        WaitNamedPipeA(pipeName, 5000);
        HANDLE hPipe = CreateFileA(pipeName, GENERIC_READ |
GENERIC_WRITE,
                                   0, NULL, OPEN_EXISTING, 0, NULL);
        if (hPipe == INVALID_HANDLE_VALUE)
PrintError("ConnectClientPipe");
        return hPipe;
    }
```

Листинг 1 – Создание и подключение к Named Pipe

Ядро алгоритма обеспечения отказоустойчивости конвейера сосредоточено в процессе Validator и приведено в листинге 2. В основном цикле приема данных узел проверяет статус каждого входящего пакета: корректные результаты с типом MSG_RESULT_OK транслируются напрямую в Collector, а при обнаружении ошибки MSG_RESULT_ERR выполняется анализ счетчика повторов. Если лимит не превышен, задание получает инкремент поля retryCount, переводится обратно в состояние MSG_TASK и перенаправляется в выделенный обратный канал повторов для повторной обработки в Processor, в противном случае сообщение сбрасывается в архив как невосстановимый сбой.

```
    while (ReceiveMessageFromPipe(hPipeIn, msg, runId)) {
        if (msg.messageType == MSG_STOP) {
            SendMessageToPipe(hPipeToCollector, msg, runId);
            break;
        }
        if (msg.messageType == MSG_RESULT_OK) {
            msg.sourcePid = GetCurrentProcessId();
            SendMessageToPipe(hPipeToCollector, msg, runId);
        } else if (msg.messageType == MSG_RESULT_ERR) {
            if (msg.retryCount < 2) {
                msg.retryCount++;
                msg.messageType = MSG_TASK;
                msg.sourcePid = GetCurrentProcessId();
                SendMessageToPipe(hPipeToProcessorRetry, msg, runId);
            } else {
                msg.sourcePid = GetCurrentProcessId();
                SendMessageToPipe(hPipeToCollector, msg, runId);
```

```
            }
        }
    }
```

Листинг 2 – Обработка ошибок и контроль лимита повторов в Validator

3.6 Результаты экспериментального исследования

При запуске комплекса все четыре процесса были успешно инициализированы с идентификатором RUN_ID=72. Генератор передал 172 задания. Процессор симулировал 59 сбоев. Валидатор успешно вернул на повторную обработку 58 заданий, которые завершились успешно со второй попытки. Задание №72 исчерпало лимит 2 повторов и было корректно сброшено в Коллектор.

Результаты выполнения процессов в консольных окнах представлены на рисунках 2, 3, 4, 5.

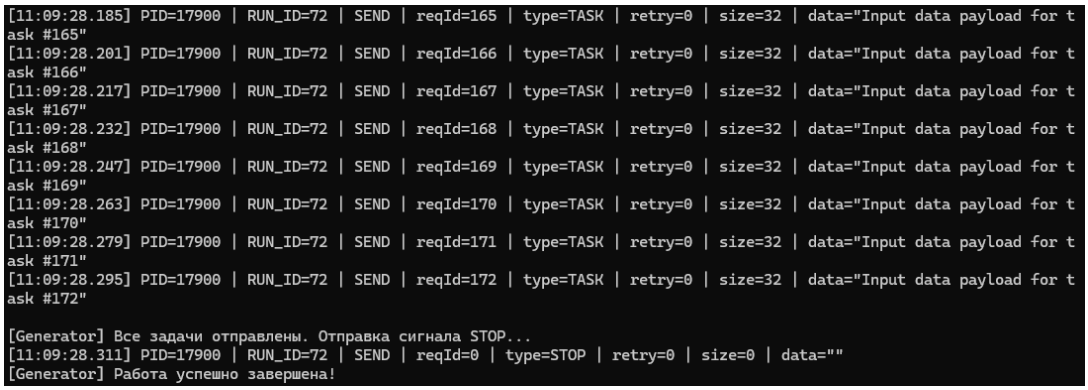


Рисунок 2 – Окно процесса Generator

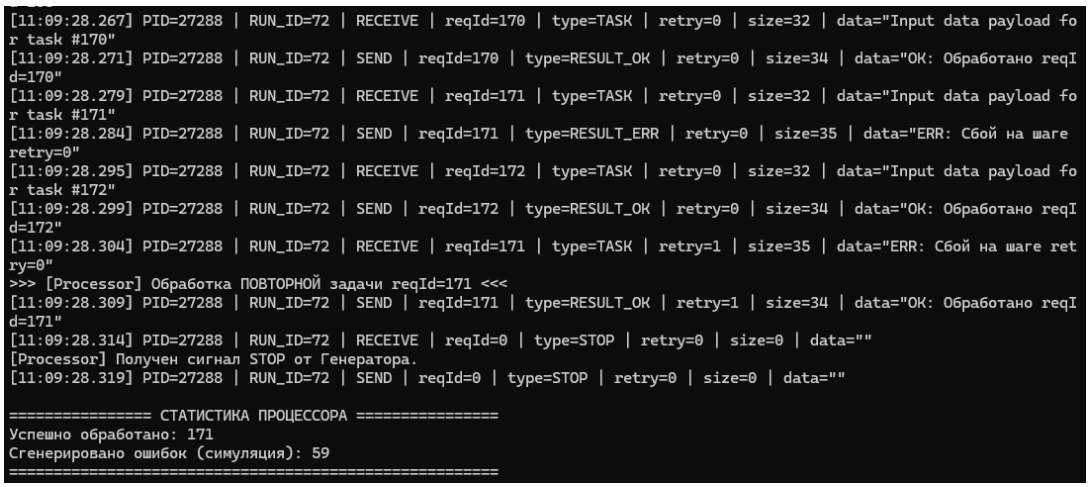


Рисунок 3 – Окно процесса Processor

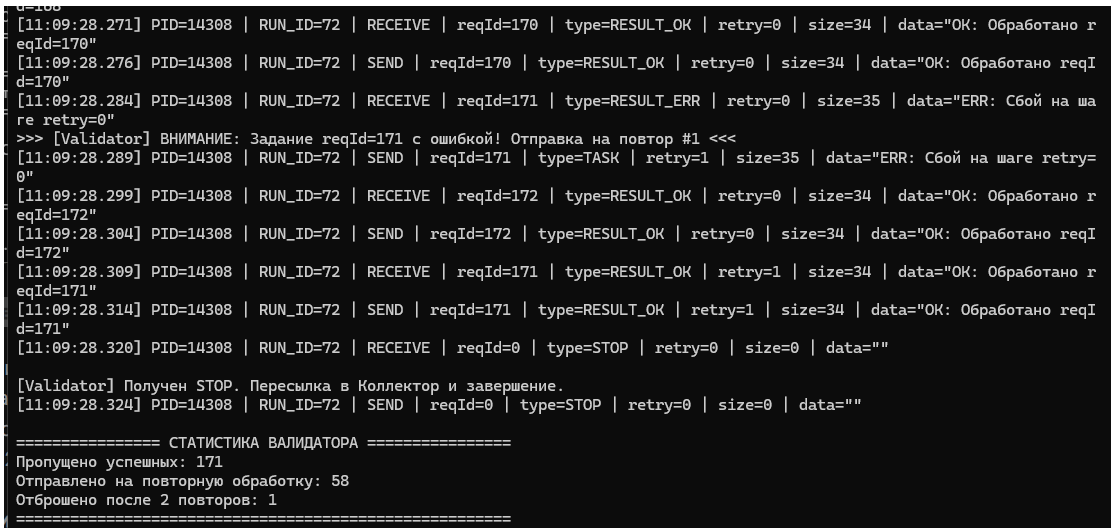


Рисунок 4 – Окно процесса Validator

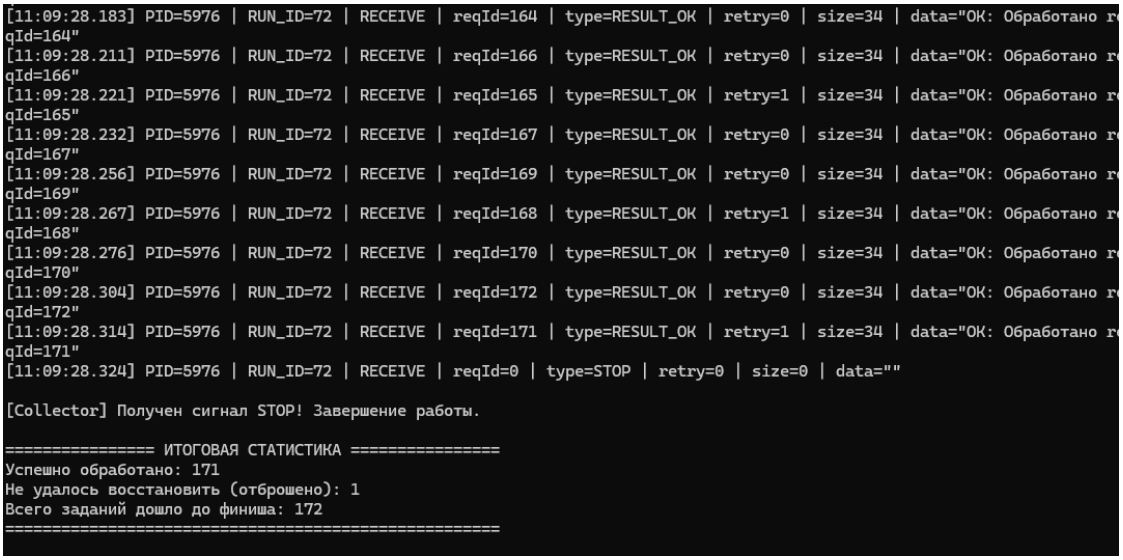


Рисунок 5 – Окно процесса Collector

Финальный баланс обработки данных в узле Collector составил:

– Успешно обработано: 171 задание;

– Не удалось восстановить (отброшено после 2 повторов): 1 задание (reqId=72);

– Всего дошло до финиша: 172 задания (потерь сообщений нет, баланс сошелся на 100%).

3.6 Результаты экспериментального исследования

В соответствии с разделом 9 методических указаний было проведено экспериментальное исследование пропускной способности реализованного механизма Named Pipe в режиме сообщений PIPE_TYPE_MESSAGE. Исследование выполнялось путем передачи непрерывной серии из 1000 сообщений для двух фиксированных объемов полезной нагрузки: малого пакета размером 64 байта и крупного пакета объемом 4 КБ (4096 байт).

Таблица 2 – Результаты измерения производительности обмена через Named Pipe

Тест	Размер сообщения (Payload)	Количество сообщений	Общее время обмена, мс	Пропускная способность, сообщ./с
1	64 байта	1000	2,02	493 291
2	4 КБ (4096 байт)	1000	3,49	286 393

Пример вывода результатов тестирования производительности в консоли приведен на рисунке 6.

```
================ ТЕСТ ПРОИЗВОДИТЕЛЬНОСТИ IPC (1000 СООБЩЕНИЙ) ================
[Тест 1 | 64 байта]: Время = 2.0272 мс | Скорость = 493291 сообщ/сек
[Тест 2 | 4 КБ]:     Время = 3.4917 мс | Скорость = 286393 сообщ/сек
================================================================================
```

Рисунок 6 – Результаты тестирования пропускной способности IPC Named Pipe

Анализ полученных данных показывает исключительно высокую производительность локального именованного канала в операционной системе Windows: обмен небольшими структурами данных (64 байта) занял всего 2,02 мс, обеспечив скорость порядка полумиллиона транзакций в секунду.

При увеличении объема передаваемых данных до 4 КБ общее время пересылки тысячи сообщений возросло до 3,49 мс, а пропускная способность составила более 286 тысяч сообщений в секунду. Снижение частоты передачи сообщений примерно на 42% при переходе к пакету в 4 КБ обусловлено физическими затратами подсистемы оперативной памяти на побайтовое копирование буферов большего объема драйвером файловой системы именованных каналов ядра Windows (NPFS).

4 ИСПОЛЬЗОВАНИЕ ИИ

В соответствии с разделом 10 методических указаний при подготовке и выполнении лабораторной работы применялась большая языковая модель Gemini (Google). Инструмент использовался исключительно в качестве справочно-консультационной системы для углубленного анализа низкоуровневых механизмов Windows API. В частности, с помощью модели было детально изучено поведение функции ConnectNamedPipe() в условиях опережающего подключения клиента со статусом ERROR_PIPE_CONNECTED, проведено сопоставление режимов передачи сообщений PIPE_TYPE_MESSAGE и потокового обмена PIPE_TYPE_BYTE, а также рассмотрена реализация неблокирующего чтения входящего буфера с помощью функции PeekNamedPipe().

Вся архитектура распределенного конвейера, топология и именование каналов с учетом индивидуального параметра STUDENT_ID=72, структура бинарного пакета протокола, логика обработки сбоев и контроля лимита повторов в узле Validator, а также непосредственная программная реализация всех модулей на языке C++20 в среде CLion были спроектированы, написаны, отлажены и протестированы студентом.

ВЫВОД

В ходе выполнения лабораторной работы были изучены механизмы межпроцессного взаимодействия (IPC) в операционной системе Windows. На практике был спроектирован и реализован отказоустойчивый конвейер из четырех изолированных процессов (Generator, Processor, Validator, Collector), взаимодействующих через двусторонние именованные каналы (Named Pipe). Был разработан собственный бинарный прикладной протокол обмена сообщениями, исключающий использование сырых указателей и обеспечивающий сквозную трассировку по идентификаторам PID, RUN_ID и requestId.

Экспериментальная проверка подтвердила полную работоспособность алгоритмов обнаружения сбоев и восстановления конвейера. Валидатор успешно вернул на повторную обработку все восстановимые сообщения (58 задач), а для фатального задания №72 зафиксировал исчерпание лимита двух повторов и изолировал ошибку без нарушения работы остальных стадий. Баланс входных и выходных сообщений в узле Collector сошелся на 100% (171 успешное и 1 отброшенное сообщение из 172 сгенерированных), что доказывает отсутствие потерь и зависаний в каналах. Исследование пропускной способности показало высокую производительность механизма Named Pipe, а штатное каскадное распространение сигнала MSG_STOP обеспечило корректное закрытие всех системных дескрипторов ядра ОС без аварийных завершений.